have our answer, yet.

Listing 6.9 clarifies these concepts by showing the implementation of this method; if you recall how insert work, it can be re-thought as a two-step operation: finding the longest common prefix of the key to be inserted already in the trie, and then adding a branch with the remaining characters: try, as an exercise, to rewrite the pseudocode for insert leveraging method `longestPrefix`.

Like the other methods we have descry bed so far, this operation is also linear in the length of the searched string: `O(m)`, if `|s|==m`.

6.2.6 Keys Matching a Prefix

The last method we are going to describe returns all the keys matching a certain prefix.

If you stop and think about the definition of a trie, the implementation for this method will flow almost naturally: even the alias for this data structure, *prefix tree*, suggests a solution. We have seen, in fact, that tries store compactly strings sharing the same prefix, because each string is translated in a path from the root to a *key node*, and strings sharing the same prefix will result in sharing the same path in the trie.

For instance, in figure 6.3, all the strings “`and`”, “`ant`”, “`anthem`” share a portion of their path, corresponding to the common prefix “`an`”.